

Software Engineering for AI

André Silva

WASP Software Engineering Course, 2026

1. Introduction

My research is focused on building better coding models. I have approached this question from multiple angles. First, by exploring methods for fine-tuning models for downstream coding tasks [1, 2], in order to obtain more effective and efficient models. Second, by creating benchmarks of realistic software engineering tasks to evaluate frontier coding agents [3, 4, 5] and studying how their performance varies from run to run [6]. Third, by studying latent program spaces and how these are and can be used to more efficiently search for programs. In gradient-based program repair [7], we showed in a controlled setting that, starting from a buggy symbolic program compiled deterministically into a differentiable numerical representation, it is possible to reach an optimum that corresponds to a correct program. In subsequent work studying the internals of coding agent models [8], we found that hidden states linearly encode whether the evolving program parses, passes tests, reduces failures, or introduces regressions. Some of these signals also encode what these properties will look like up to 25 agent steps ahead, suggesting that some form of latent program search is already happening inside agentic models.

2. Principles from Robert's lectures (Verification, Validation, Oracle Problem)

One of the ideas discussed during the course is the difference between verification and validation. Verification is the process of checking that the system we are building meets its requirements, while validation is the process that checks that those requirements are indeed the right ones for the problem we are trying to solve. This distinction was always important in software engineering, and is today more relevant than ever.

From the perspective of software engineering, implementation (i.e., translating requirements into code) used to be the most expensive part of the process, requiring time and effort from highly skilled engineers. Today, the majority of code is not written by humans but rather generated by coding agents ¹. The impact of this change, particularly as coding agents become cheaper and more capable, is that the cost of implementation is no longer the bottleneck in software development. And, while maybe not immediately true, it is reasonable to expect that the cost of verification will also decrease as trust in coding agents increases and as we develop better ways to verify their implementations, particularly through automated testing (traditional or AI-driven) and via formal methods. What is not clear, at least to me, is how a much softer, ambiguous, and human-centered process like validation will evolve and, particularly, scale with the increasing use of coding agents.

From the perspective the development of coding agents, and thus my research, the distinction between verification and validation is also important. Today, the way we learn models that

¹<https://www.businessinsider.com/google-ai-generated-code-75-gemini-agents-software-2026-4>

can act as coding agents is by collecting and creating large amounts of software engineering tasks in the form of reinforcement learning environments. These environments are used, during post-training, to simulate thousands of attempts and iteratively improve the models performance from learnt lessons. One of the main challenges in this process is in determining whether each simulation was actually successful, as this needs to be done in an automated and scalable way. This means, for example, a software engineering tasks would be evaluated by checking if the code passes a set of tests (verification) but not necessarily by checking that the code actually solves the problem that was intended to be solved (validation). The problem is that reinforcement learning is famously prone to reward hacking, which occurs when models find a way to maximize their rewards without actually solving the intended problem, and we know that the gap between verification and validation is a fertile ground for reward hacking [9, 10].

For this reason, I believe that developing better ways to verify that software, whether it be traditional software or coding agents, does what it is supposed to do is one of the most important challenges in AI. If the verification used during post-training is weak, models will learn to exploit it. From the software engineering perspective this could mean having coding agents that eagerly announce they have completed a task while the code is actually written in an odd way pass verification but does not actually solve the problem. From the AI safety perspective, this means that cheating behaviors learnt in environments with weak verification generalize, leading to scenarios like the OpenAI/HuggingFace incident [11] where humans take months to realize that misaligned behaviors learnt during post-training due to bad verification have lead to misaligned agents running loose.

3. Principles from the guest lectures

A recurring theme in the guest lectures was again about the shift towards humans handling requirements more than the codes, from the same angle that I discussed in the previous section. Another aspect that was also discussed is how AI systems also need to be developed with the same principles of software engineering. For example, Per Lendberg painted an analogy between requirements, checklists, examples and the information we provide AI agents as prompts. Engineers are likely going to spend much less time reviewing code, which will be generated and validated as automatically as possible, and much more time on designing the requirements and checklists that are used to ensure the system is doing what it is supposed to do.

One research area parallel to mine studies how we can optimize prompts and agent harnesses to improve the performance of AI agents [12], using the same kind of data we already use to learn models but specialized to the task the agent is going to perform in practice. I believe this is an important area of research, and likely one that will continue to grow in importance in the future as more and more custom AI agents are build, deployed in production, and continuously improved.

4. Data scientists, software engineers, and AI engineers

The book describes data scientists as using exploratory, science-like workflows centered on data, model construction, and held-out predictive performance. Software engineers instead focus on the whole product from requirements, architecture, implementation, deployment, maintenance, and trade-offs like usability, scalability, security, time, and cost. I agree that this is a useful description but one that is changing fast and likely not going to last for very long.

It is reasonable to expect that the majority of traditional data scientist tasks, like iterating on data and models based on model-centric performance metrics, will be increasingly done by AI

agents as this kind of job fits perfectly with the type of tasks that can be automatically verified and thus used for post-training of agentic models. As a consequence, I expect data scientists to move to higher-level abstraction levels which incorporate a more holistic view of the system, treading them closer to the domain of software engineering and towards activities that are harder to automate, like ensuring that the system is actually solving the right problem, and that it is doing so in a way that is aligned with human values and expectations.

From the software engineering perspective, the increasing use of AI models as components of software systems means that software engineers will need to understand the behavior of these models, which are often stochastic, and rely on tools that are more commonly used by data scientists, like statistical analysis, to understand and debug the behavior of these models and the systems built around them. It is, for that reason, reasonable to expect that software engineering will change towards what some people call AI engineering. For example, in traditional software engineering we rely heavily on testing, from unit tests to integration tests, that gives us a more binary label about whether the system is working as intended or not. Now, we will start seeing benchmarks and their non-binary outcomes become very important tools that instead of answering whether the system is working or not tell us to what degree it is working.

5. Paper analysis

The Saving SWE-Bench paper [13] from Microsoft, published at CAIN 2026, looks at the discrepancy between some of the most popular software engineering benchmarks like SWE-Bench [14] and the way developers actually interact with chat-based coding agents. This comparison is focused primarily on the instructions given to the agent. Through an analysis of telemetry data and pattern extraction, the authors find that communication styles are significantly different from those in the problem statements of existing benchmarks, and that users typically provide much less information. For example, users rarely provide information about how to reproduce a certain behavior, the difference between expected and actual behavior, information about the environment, and are much more concise in their queries. Then, to bridge the gap between a given benchmark and telemetry data, the authors propose a mutation approach that transforms problem statements into the typical style of user queries. This way, mutated benchmarks are more likely to reflect real-world performance of coding agents. This paper is directly related with my research on coding benchmarks ².

Another paper that is directly related with this one is SWE-Universe [15] from the Qwen Team at Alibaba. In this paper, they propose a scalable framework for automatically constructing software engineering reinforcement learning environments from GitHub pull requests. They show that, using their framework, they are able to scale the number of environments to 807,693, an order of magnitude larger than publicly known benchmarks. Then, they use this data to improve train a model to 75.3% score on SWE-Bench Verified. This paper is also directly related with my research on automatically building coding datasets, and overall in how we can improve coding models.

Now, let's consider we want to train an agentic model and build a coding agent harness around it that are capable of automating all coding tasks. There are good incentives for doing so. First, software is a big market. Some figures put worldwide IT spending at \$6 Trillion ³ and growing. A big part of that is software and salary spending. Second, software is the native way for AI

²Small disclaimer, I actually interned with this team and worked them on another effort to expand benchmark coverage.

³<https://www.gartner.com/en/newsroom/press-releases/2025-10-22-gartner-forecasts-worldwide-it-spending-t>

agents to interact with computers. It is through software that agents can do things like read and edit documents, browse the internet, communicate with people and other agents, make purchases, etc. It is through software that agents are able to perform any type of white collar job. The TAM of salaries for white collar jobs is even larger than that of software alone. This is exactly what leading AI labs are doing.

It is commonly believed by many people working in AI that the current generation of algorithms are enough to automate all kinds of white collar work ⁴. In other words, the only thing missing is enough coverage. To have reinforcement learning environments for most types of white collar tasks. If we do so, existing algorithms are enough to ingest them and output an agentic model capable of performing any white collar task.

Now going back to Saving SWE-Bench and SWE-Universe. Both of these papers, albeit in different ways, address this coverage problem. Saving SWE-Bench diversifies problem statements, taking the original ones and making them closer to what users ask coding agents today. They do this in the context of benchmarks, but the process is the same for building reinforcement learning environments. This is one angle, to diversify the way we instruct, prompt, command agents. The more ways of interacting with them we cover, the more adaptable models will be to production scenarios. SWE-Universe focuses on collecting as many real-world cases as possible by deploying an agent to work around the common problems that arise when collecting real-world data: how to build a given project? how to verify that the task is completed? how to check that the problem is consistent and complete? By being able to collect more real-world scenarios, they are able to cover more types of task, more programming languages, more application domains, more cultural aspects.

This is directly related with my research. Like in AI engineering, the first step towards building a system, a coding agent in this case, is to be able to define and measure the goal. In our scenario and in my research, we want to be able to define the kind of tasks the agent should be able to complete, be able to measure how well it is at doing so, and finally use them for training. When building GitHub-Actions [3] our main goal was to find ways to automatically build more complex and realistic software engineering benchmarks, focusing on how to leverage existing CI configurations to automatically build both the verifiers and the sandboxes for the environments. In another paper, “On Randomness in Agentic Evals” [6], we looked at how randomness affects measured performance and how we can use tools from statistics to ensure we size our evaluations in a way that our measurements are meaningful.

Generally speaking, the main takeaway from this is that, analogous to coverage in traditional software testing, where we want to cover code surface, execution paths, stress test systems under different loads, to be able to test (and train) AI systems we need to cover as many of the intended usecases as possible too. That is the only way we have to increase confidence in the systems, by providing meaningful statistical guarantees, and to improve systems such that we increase rates of success.

6. Evidence, originality, and GenAI-use transparency

The main sources were the course slides, Chapters 1 and 2 of Kästner’s book, and the referenced papers. The references include information about where each paper is published or available as pre-print. I also relied on miscellaneous sources such as podcasts, tweets, newsletters, blogposts,

⁴For example, Sholto Douglas, a researcher at Anthropic, claimed that “the current suite of algorithms are sufficient to automate white collar work provided you have enough of the right kinds of data” on the Dwarkesh podcast last year <https://www.dwarkesh.com/p/sholto-trenton-2>.

and similar media whose information I have digested throughout the last few years. I place greater trust on sources which provide more technical details, particularly those who have been reproduced by third-parties. This is typically more common in the case of peer-reviewed paper but not necessarily the case for all. Moreover, frontier AI research is not made publicly available as research papers in many cases, so other kinds of sources like podcasts, blogposts, or even tweets do often carry useful and, depending on the person behing it, trustworthy information.

I used Codex to assist with finding and ranking papers from CAIN by how related they were to my research. I also used the same tool to check the consistency and completeness of my writing.

References

- [1] A. Silva, J. F. Ferreira, H. Ye, and M. Monperrus, “Mufin: Improving neural repair models with back-translation,” *arXiv preprint arXiv:2304.02301*, 2023.
- [2] A. Silva, S. Fang, and M. Monperrus, “Repairllama: Efficient representations and fine-tuned adapters for program repair,” *IEEE Transactions on Software Engineering*, 2025.
- [3] N. Saavedra, A. Silva, and M. Monperrus, “Github-actions: Building reproducible bug-fix benchmarks with github actions,” in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pp. 1–5, 2024.
- [4] A. Silva, N. Saavedra, and M. Monperrus, “Github-java: A reproducible benchmark of recent java bugs,” in *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, pp. 118–122, IEEE, 2024.
- [5] A. Silva and M. Monperrus, “Repairbench: Leaderboard of frontier models for program repair,” *2025 LLM4Code The Second International Workshop on Large Language Models for Code*, 2025.
- [6] B. H. Bjarnason, A. Silva, and M. Monperrus, “On randomness in agentic evals,” in *ICLR 2026 Agents in the Wild: Safety, Security, and Beyond*.
- [7] A. Silva, G. Thorén, and M. Monperrus, “Gradient-based program repair: Fixing bugs in continuous program spaces,” *arXiv preprint arXiv:2505.17703*, 2025.
- [8] A. Silva, H. Tu, and M. Monperrus, “Latent programming horizons in coding agents,” *arXiv preprint arXiv:2607.05188*, 2026.
- [9] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.
- [10] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, “Is the cure worse than the disease? overfitting in automated program repair,” in *Proceedings of the 2015 10th joint meeting on foundations of software engineering*, pp. 532–543, 2015.
- [11] METR, “Brief independent investigation of agents’ behavior, reasoning and collaboration in the openai / hugging face hacking incident.” <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>, 08 2026.
- [12] L. A. Agrawal, S. Tan, D. Soylu, N. Ziemis, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, *et al.*, “Gepa: Reflective prompt evolution can outperform

- reinforcement learning,” in *International Conference on Learning Representations*, vol. 2026, pp. 8479–8565, 2026.
- [13] S. Garg, B. Steenhoeck, and Y. Huang, “Saving swe-bench: A benchmark mutation approach for realistic agent evaluation,” *arXiv preprint arXiv:2510.08996*, 2025.
- [14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?,” in *International Conference on Learning Representations*, vol. 2024, pp. 54107–54157, 2024.
- [15] M. Chen, L. Zhang, Y. Feng, X. Wang, W. Zhao, R. Cao, J. Yang, J. Chen, M. Li, Z. Ma, *et al.*, “Swe-universe: Scale real-world verifiable environments to millions,” *arXiv preprint arXiv:2602.02361*, 2026.